ZipMould

A slime-mould-inspired solver for Zip puzzles

Gabriel Mitelman Tkacz · May 2026

From continuous metaheuristic to discrete combinatorial search — porting **Li et al. (2020)** to a Hamiltonian-path domain.

Three acts, twenty minutes

Act 1 · Li (≈7 min)

What is the **Slime Mould Algorithm**? The biology, the equations, the empirical case.

Act 2 · ZipMould (≈7 min)

Porting continuous SMA to a **discrete combinatorial** problem. Where the analogy holds, where it breaks, what we changed.

Act 3 · Demo (≈5 min)

Live trace replay in the **Vue 3 visualiser**.

The bridge in one line

Li's update

$$X(t+1) = v_c \cdot X(t) + v_b \cdot (W \cdot X_A - X_B)$$

becomes ZipMould's edge update

$$\tau \leftarrow v_c \cdot \tau + v_b \cdot \Delta_{\text{rank-weighted}}$$

What is *Physarum polycephalum*?

- **Acellular slime mould.** Single multinucleate organism, no nervous system.
- Forages by extending a **venous network** through a substrate.
- **Positive feedback**: high food concentration → faster cytoplasmic flow → vein thickens.
- **Negative feedback**: starved branches retract.
- Demonstrated to solve mazes, approximate **Tokyo's rail network**, and behave like a distributed optimiser without any centralised control.

> The algorithm we'll see formalises three observed behaviours: **approaching food**, **wrapping food** (the venous-thickness feedback), and **grabbling food** (the bio-oscillator amplitude).

Li, Chen, Wang, Heidari & Mirjalili (2020), Future Generation Computer Systems 111, 300–323. Tero et al. (2010), Science 327, 439–442.

SMA Eq. (2.1) — *approach food*

$$\vec{X}(t+1) = \begin{cases} \vec{X}_b(t) + \vec{v}_b \cdot \left(\vec{W} \cdot \vec{X}_A(t) - \vec{X}_B(t)\right), & r < p \\ \vec{v}_c \cdot \vec{X}(t), & r \geq p \end{cases}$$

- $\vec{X}_b$ — best individual found so far.
- $\vec{X}_A, \vec{X}_B$ — two random individuals (provides exploration vector).
- $\vec{W}$ — fitness-rank-derived **weight** (Eq. 2.5, next slide).
- $\vec{v}_b \in [-a, a]$, where $a = \operatorname{arctanh}(1 - t/T)$.
- $\vec{v}_c$ decreases linearly from 1 to 0.
- $p = \tanh|S(i) - DF|$ — adaptive switching threshold.

Li et al. (2020), §2.3.1.

SMA Eq. (2.5) — the weight $\vec{W}$

$$W_i = \begin{cases} 1 + r \cdot \log\left(\dfrac{bF - S(i)}{bF - wF} + 1\right), & i \in \text{top half (good fitness)} \\ 1 - r \cdot \log\left(\dfrac{bF - S(i)}{bF - wF} + 1\right), & i \in \text{bottom half (poor fitness)} \end{cases}$$

- **Top half** of population pulls toward favourable areas — **positive feedback**.
- **Bottom half** is pushed away — **negative feedback** simulating retraction of starved veins.
- $\log$ tames the rate of change; $r \sim \mathcal{U}(0, 1)$ keeps the response stochastic.
- Captures the slime mould "preference" via **fitness ranking**, not absolute fitness.

Li et al. (2020), §2.3.2 — "Wrap food" mathematical model.

SMA Eq. (2.7) — the full update

$$\vec{X}^* = \begin{cases} \text{rand} \cdot (UB - LB) + LB, & \text{rand} < z \\ \vec{X}_b(t) + \vec{v}_b \cdot (W \cdot \vec{X}_A - \vec{X}_B), & r < p \\ \vec{v}_c \cdot \vec{X}(t), & r \geq p \end{cases}$$

- Three branches, **per individual, per iteration**:
  i. **z-branch** (prob $z = 0.03$ in Li): random restart in the search box.
  ii. **Approach** branch (prob $\approx p$): exploit best with W-weighted exploration.
  iii. **Oscillate** branch: shrink toward origin with v_c
  .

The z-branch is what lets SMA *escape local optima* without explicit niching or restarting the whole population.

Li et al. (2020), §2.3.2 Eq. (2.7); z chosen as 0.03 from sensitivity sweep §3.4.

The v_b / v_c schedule

v_c — linear contraction

$$v_c \in [-1, 1], \quad v_c \to 0 \text{ as } t \to T$$

Smoothly damps the *oscillate* branch. Late in the run, $v_c \cdot X \approx 0$: the agent stops drifting on its own.

v_b — saturating amplitude

$$v_b \in [-a, a], \quad a = \operatorname{arctanh}(1 - t/T)$$

Early: $a \to \infty$ → big jumps. Late: $a \to 0$ → fine-grained exploitation.

Combined effect

Early iterations: **exploration** dominates via large v_b.

Late iterations: **exploitation** dominates as both v_b and v_c shrink.

The slime mould "decides whether to approach the current source or seek another" — encoded as oscillation amplitude.

Li et al. (2020), §2.3.3 "Grabble food", Fig. 5.

SMA — the algorithm

```
INITIALISE population X_1 ... X_n at random in [LB, UB]
FOR t = 1 ... T:
    evaluate fitness S(i) for all i
    sort population, identify bF, wF, X_b
    compute W via Eq. (2.5)        # rank-based positive/negative weights
    FOR each individual i:
        sample r ~ U(0,1), rand ~ U(0,1)
        update v_b, v_c, p          # schedule
        IF rand < z:
            X_i ← random restart in [LB, UB]
        ELIF r < p:
            X_i ← X_b + v_b * (W * X_A - X_B)   # approach
        ELSE:
            X_i ← v_c * X_i                      # oscillate
RETURN bF, X_b
```

- One outer loop, three inner branches, no derivatives, no gradients.
- Five hyperparameters total: population n, iterations T, restart prob z, plus the schedule constants baked into v_b, v_c.

Why the field cared

Empirical case

- **23 classical benchmarks** (unimodal + multimodal) + **10 CEC2014** functions: SMA wins or ties first on the majority.
- Outperforms WOA, GWO, MFO, BA, SCA, PSO, SSA, MVO, ALO on most multimodal cases.
- **4 engineering design problems** (welded beam, pressure vessel, cantilever, I-beam): best feasible solution on all four.
- Convergence curves show **fast early descent + accurate late refinement**.

Why it works

- W implements an explicit **diversity term** — bottom-half repulsion prevents premature convergence.
- v_b schedule provides **automatic exploration→exploitation** transition, no operator scheduling.
- z-branch escape is **simple but effective** for getting out of local basins.

Li et al. (2020), Tables 5–22; Figs. 9–14 (convergence curves).

The Zip puzzle

Formal definition

Given a grid $G_{N \times N}$ with:

- K ordered waypoints $w_1, w_2, \ldots, w_K$
- A set of **walls** (forbidden edges between adjacent cells)
- A set of **blocked** cells

Find a **Hamiltonian path** $\pi_1, \ldots, \pi_L$ (where $L = N^2 - |\text{blocked}|$) such that:

1. consecutive cells are 4-adjacent and not wall-separated,
2. waypoints appear in **ascending order**,
3. $\pi_1 = w_1$ and $\pi_L = w_K$.

LinkedIn's daily Zip puzzle popularised the format.

The decision problem is **NP-complete** (reduces to Hamiltonian-path-with-pinned-vertices), but tractable in practice for $N \leq 10$ — which is exactly where metaheuristics earn their keep.

Why continuous SMA can't transfer directly

Li's SMA lives in $\mathbb{R}^d$

- $\vec{X}_A - \vec{X}_B$ is a Euclidean direction vector.
- $v_b \cdot W$ scales an amplitude in continuous space.
- Step is just vector addition.

Zip lives on a graph

- "Position" is a partial Hamiltonian path, not a coordinate.
- $\vec{X}_A - \vec{X}_B$ is **undefined** between two paths.
- The natural state is **edge usage**, not point coordinates.

The bridge

Borrow **stigmergy** from **Ant Colony Optimisation** (Dorigo, 1992): pheromone τ on edges plays the role of the agent state.

Then port SMA's **update dynamics** — the v_b /v_c schedule, signed rank weights, and z-restart — onto the pheromone, not onto a coordinate.

ZipMould — pipeline

```
┌──────────────┐    ┌──────────────────┐    ┌──────────────────┐
│ Puzzle       │    │ Feasibility      │    │ Solver kernel    │
│ (grid + K    │ ─▶ │  precheck        │ ─▶ │  (Numba @njit)   │
│ waypoints)   │    │ O(N²)            │    │ population × T   │
└──────────────┘    └──────────────────┘    └──────────────────┘
                                                   │
                                                   ▼
                                             ┌──────────────────┐
                                             │ Fitness +        │
                                             │ CBOR trace       │
                                             └──────────────────┘
```

Feasibility checks (cheap, decisive)

- Waypoint reachable & not blocked
- Free subgraph **connected** (BFS from w_1 covers all free cells)
- **Parity bound**: $|F_0 - F_1| \leq 1$ on chessboard colouring
- **Endpoint parity** consistent with w_1, w_K

Failing any of these proves no Hamiltonian path exists — we skip the kernel entirely.

Step 1 — ACO-style construction

Each walker, at each cell, picks a 4-neighbour by **softmax over (pheromone + heuristic)**:

$$P(c \to c') \propto \exp\Big(\alpha \cdot \tau_{cc'} + \beta \cdot \log \eta_{c'}\Big)$$

with combined heuristic $\eta_{c'} = \mathrm{softplus}(h_m)^{\gamma_m} \cdot \mathrm{softplus}(h_w)^{\gamma_w} \cdot \mathrm{softplus}(h_a)^{\gamma_a} \cdot \mathrm{softplus}(h_p)^{\gamma_p}$.

Heuristic	Role	Source
h_m — Manhattan	Pull toward the next waypoint	$-d_M(c', w_{\mathrm{seg}+1})$
h_w — Warnsdorff	Prefer low-degree cells; visit dead ends first	classical knight's-tour heuristic (1823)
h_a — Articulation	Reject moves that **disconnect** the unvisited free subgraph	flood-fill check
h_p — Parity	Maintain $\lvert F_0 - F_1 \rvert \leq 1$ post-move	chessboard invariant

Combined multiplicatively after softplus to admit any-sign components; α = 1, β = 2 defaults match ACO conventions (Dorigo & Stützle, 2004).

Step 2 — SMA-style pheromone update

```
# src/zipmould/solver/_kernel.py — _pheromone_update
progress = float(t) / float(T)
v_b = math.tanh(1.0 - progress)        # Li-inspired, BOUNDED (cf. arctanh)
v_c = 1.0 - progress                   # Li 2.4 verbatim

# Signed rank weights ∈ [-1, +1]:  best walker = +1, worst = -1, median ≈ 0
denom = float(n - 1)
weights[i] = (float(n) - 2.0 * float(r) + 1.0) / denom

# Per-edge update — the ZipMould analogue of Li Eq. (2.7)
new_val = v_c * tau[s, e] + v_b * deposit[s, e]

# Li z-branch escape — verbatim, on edges
if z > 0.0 and np.random.random() < z:
    new_val = np.random.normal(0.0, tau_max / 4.0)
```

> The **signed** rank weight is our discrete analogue of Li's W: top-half walkers *deposit* pheromone, bottom-half walkers *evaporate* it on the same edges. Without sign, this collapses to vanilla ACO.

What changed from Li, and why

Li (2020) — continuous	ZipMould — discrete	Why the change
State $\vec{X} \in \mathbb{R}^d$	Pheromone $\tau \in \mathbb{R}^m$ (m = #edges)	No coordinate space; edges carry memory
$W_i = 1 \pm r \log(\cdot)$	$W_i = (n - 2r + 1)/(n - 1)$	Linear rank — bounded, no $\log$ singularity
$v_b \in [-a, a]$, $a = \operatorname{arctanh}(1 - t/T)$ — **unbounded** at $t = 0$	$v_b = \tanh(1 - t/T)$ — bounded in $[0, \tanh 1]$	Discrete deposits diverge under unbounded v_b; saturation stabilises
Three-branch *switching* update (z / approach / oscillate)	**Single** sum $v_c\tau + v_b\Delta$ + z-branch noise	All three Li ingredients per step; no per-individual branch lottery
$X_A - X_B$ random direction	Replaced by **rank-weighted aggregate** $\sum_w W_w \cdot 1[\text{walker } w \text{ used edge } e]$	"Difference of two random points" undefined on graph

These are not improvements — they are **adaptations** that preserve Li's mechanism (positive/negative feedback + amplitude schedule + restart escape) under the constraints of a discrete edge-pheromone state.

Ablation matrix

Two design knobs

- **Pheromone mode**:
  - `unified` — one τ per edge, shared across the full path
  - `stratified` — one τ per (edge, inter-waypoint segment) pair
- **Sign**:
  - `signed` — both attract & repel (full SMA analogue)
  - `positive` — only top half deposits (closer to ACO)

4 conditions × 4 baselines

	unified	stratified
signed	A	B
positive	C	D

Pre-registered hypotheses

1. **signed** > **positive** on hard puzzles (negative feedback breaks symmetry)
2. **stratified** > **unified** when K is large (per-segment memory matters)

The Numba kernel — why this runs

```
@nb.njit(cache=True)
def _walker_step(walker_id, pos, visited, ..., tau, alpha, beta_log, ...):
    for d in range(4):                                  # 4 neighbours
        nb_cell = adjacency[cur, d]
        if nb_cell < 0 or _bit_test(visited, walker_id, nb_cell): continue
        h_a = h_articulation(walker_id, nb_cell, visited, adjacency, ...)
        if h_a == NEG_INF: continue                     # disconnects subgraph
        logits[d] = alpha * tau_val + beta_log * math.log(eta)
    # softmax with log-sum-exp; roulette sample; set visited bit; update parity
```

- **Bitset** `visited` (uint64 words) → O(1) membership, cache-friendly.
- `@njit` JIT-compiles the hot loop to **C-speed**; pure-Python is ≈100× slower.
- Articulation flood-fill is the heaviest inner cost — `work_stack` shared across walkers.

Baselines + statistical protocol

Baseline	Pheromone	Deposit	Notes
aco-vanilla	unsigned, unified	$\propto$ fitness	Classical ρ-evaporation, no restart noise
heuristic-only	none	—	Greedy on η alone — measures heuristic strength
random-walk	uniform	none	Pure exploration floor
backtracking	n/a	n/a	Exhaustive DFS with parity + articulation pruning

Pre-registered protocol

- **Train / dev / test** splits, stratified by puzzle hardness (computed offline from BFS depth + K/L).
- **Held-out test set**: results computed *once*, after design freeze.
- **McNemar paired test** on solve / no-solve outcomes per puzzle.
- **Benjamini-Hochberg FDR correction** across the 4×4 condition matrix.
- All seeds reproducible via `derive_kernel_seed(global_seed, run_seed, puzzle_id, config_hash)`.

Visualiser — how the trace becomes a movie

```
zipmould solver
    │
    ▼
CBOR trace (per-frame snapshot)
    • t, v_b, v_c
    • walker positions + segments
    • tau delta (sparse)
    • best fitness so far
    │
    ▼
FastAPI server (uv run zipmould viz serve)
    │  HTTP + cbor-x stream
    ▼
Vue 3 + Pinia + Tailwind 4
    • GridCanvas (SVG)
    • FitnessChart (Chart.js)
    • WalkerTable
    • Frame scrubber
```

Why CBOR

- **Sparse** τ-deltas only — frames stay small even on 200-iter runs.
- Schema-light, **streamable**, native binary in `cbor-x`.

Why client-side replay

- Solver is heavy; we run it **once** to disk, then scrub instantly in the browser.
- Decouples experiment runs from presentation/inspection.

Live demo

Switching to the visualiser…

```
# Terminal 1 — solver/HTTP backend
uv run zipmould viz serve

# Terminal 2 — Vue dev server
cd viz-web && bun run dev
```

What we'll show: load a trace · play it back · toggle τ heatmap and walker layers · scrub to a specific iteration · compare *signed* vs *positive* conditions on the same puzzle.

Takeaways + future work

Takeaways

- Li's SMA is a **mechanism**, not a code path: W + v_b/v_c + z-branch each have a discrete analogue.
- Stigmergy (ACO) supplies the missing state space; SMA supplies the **dynamics on top of it**.
- Bounding the v_b schedule is the **single largest deviation** — and forced by discreteness.
- Pre-registered ablation matrix lets us attribute any win to mechanism, not tuning.

Future work

- **Warm-start** from the heuristic-only baseline path.
- **Learned heuristic** η (small MLP on local cell features) — drop in beside Manhattan/Warnsdorff/etc.
- **Multi-objective**: shortest path under a tight wall budget.
- **Beyond Zip**: knight's tours, generalised Hamiltonian-path-with-pinned-vertices.

Thanks

Questions, sceptical or otherwise, welcomed.

References

- **Li, S., Chen, H., Wang, M., Heidari, A. A. & Mirjalili, S.** (2020). Slime mould algorithm: A new method for stochastic optimization. *Future Generation Computer Systems* 111, 300–323.
- **Dorigo, M. & Stützle, T.** (2004). *Ant Colony Optimization*. MIT Press.
- **Mirjalili, S. & Lewis, A.** (2016). The Whale Optimization Algorithm. *Advances in Engineering Software* 95, 51–67.
- **Tero, A. et al.** (2010). Rules for Biologically Inspired Adaptive Network Design. *Science* 327, 439–442.
- **Warnsdorff, H. C.** (1823). *Des Rösselsprungs einfachste und allgemeinste Lösung*.
- **McNemar, Q.** (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika* 12, 153–157.